43008: Reinforcement Learning

Assignment 3, Part E Project Report

Project Title: Train Schedule Optimisation under Disruptions

Team Name and Project Number: RailTech (Project #27)

Team Members: Daniel James Martirosov (24948933), Ziheng Deng (25595315), Laurean Lim (25049863)

Date: 25 October 2026

Table of Contents

Abstract....3
1. Introduction and Background....3
1.1 The problem you tried to solve....3
1.2 Motivation....3
1.3 Application....3
1.4 Dataset/Environment....4
2. Overview of the architecture/system....5
2.1 Flow Diagram and Workflow....5
2.2 System Architecture Design....6
2.3 GUI/Environment Design....7
3. Results and Evaluation....9
3.1 Experimental Settings....9
3.2 Pre-Processing....9
3.3 Experimental Results....9
3.4 Limitations....13
4. Discussion and Conclusions....15
5. References....15
Appendices....17
A. Individual Contribution....17
B. Individual Contribution Split....17

Abstract

We ask whether a learned policy can beat a schedule-following controller when a track becomes unavailable. Two Flatland scenarios are built from the October 2026 TfNSW GTFS feed: a four-track Central-Strathfield corridor with 52 scheduled movements and the Granville-Harris Park junction with 126. The corridor is disrupted by an exogenous blockage that removes a cell's rail transitions for 400 steps, so a policy cannot make the incident vanish by rerouting. Under that regime, and across three algorithms, only the warm-started PPO learns to change track: it improves the metric from -0.0662 to -0.0005, a paired gain of +0.0657 on 30 held-out seeds (30 of 30, $p < 1e-4$) with every train completing, and on a clean network it scores 0.0000 with no crossing at all. A2C never learns the crossing and DQN parks half the fleet. The junction remains a loss: the selected checkpoint dispatches and jams (-0.2400, 59 of 126 trains home) instead of parking, and still finishes behind the scripted route follower (-0.1823).

1. Introduction and Background

1.1 The problem you tried to solve

A suburban rail corridor is a scheduling problem wrapped in a control problem. The timetable is fixed, the track layout offers few alternatives, and the failures that matter remove a cell of track for longer than the headway behind it. Classical rescheduling solves this with a central optimiser that sees the whole fleet. Our question is narrower: with one shared policy controlling every train from its own local observation, can a learner discover that it should change track before a blockage, and can that behaviour be measured honestly?

The measurement is half the problem. Flatland's metric charges lateness, missed stops and cancellations, and a policy can score well by cancelling trains rather than running them. We keep that metric as the objective and report a second statistic, delay_all, that charges every agent its arrival minus its scheduled arrival using the episode end for a train that never arrives, so a collapsed policy cannot look punctual by dropping unfinished trains from a survivors-only mean.

1.2 Motivation

Sydney's network loses whole corridors to signal repairs, overhead wiring and incidents, and the recovery decisions are made per train by whoever is watching the panel. A policy that learns the local rule - change track at this crossover when that cell is blocked - is exactly the shape of decision a signaller makes, and simulation is the only place to measure it repeatably. Reinforcement learning fits because the value of a diversion depends on what every other train does next, which no schedule can know in advance.

1.3 Application

The project delivers a simulation-based pipeline rather than a deployed controller: two scenarios built from the real timetable, a shared per-train policy, and an evaluation layer that scores any arm against a scripted baseline on the same seeds. Everything runs from one notebook (the graded artefact) and the command-line tools in scripts/, which execute that notebook's own cells so the CLI and the notebook cannot diverge.

1.4 Dataset/Environment

The data is the TfNSW GTFS static feed (about 1.4 GB), filtered to the Thursday 15 October 2026 service between 16:00 and 19:00, with link traversal times derived from the shapes rather than from straight-line distance. Two Flatland levels are built from it: a condensed four-track Main Suburban corridor and the Main Western junction at Granville, where the Suburban pair and the Old Main South converge at grade. Flatland 4.3.0 supplies the grid, the switches, the crossovers and the metric.

Item	Corridor	Junction
Scenario	network-simple-compact	granville-harris-park
Level	87 x 8, four through tracks	20 x 20 Y-link junction
Movements	52	126
Episode budget	999 steps	757 steps
Disruption	2 exogenous blockages, 400 steps	3 train-borne failures, 220 steps
Observation	tree depth 3, radius 50, plus 12 features (1032)	same
Action	Discrete(5) per train	same
Held-out seeds	3000 to 3029	3000 to 3029 (legacy rows); 3001, 3002 (current)
Baseline	forward-only reflex	scripted route follower

Table 1: Experimental settings. The maps share the observation, action space and evaluation protocol; they differ in disruption mechanism because only the corridor has a switchable pair of parallel tracks.

2. Overview of the architecture/system

2.1 Flow Diagram and Workflow

The workflow runs in seven stages, each with a defined input and output (Figure 1). The dashed arrow is deliberate: every change to the observation, reward or disruption regime was re-validated by re-running the same paired evaluation, so the report only contains numbers produced by the final configuration, with the two labelled junction rows of Table 6 as the one disclosed exception.

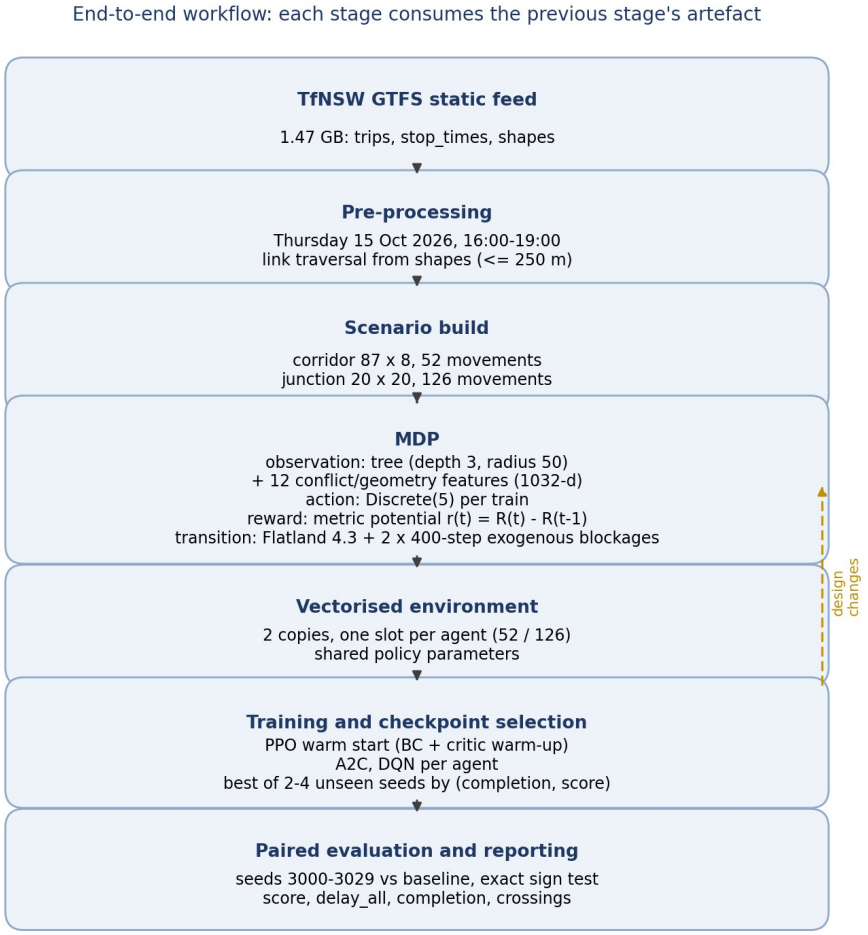


Figure 1: End-to-end workflow. Each stage consumes the artefact produced by the previous one; the dashed arrow marks the design loop that re-ran the evaluation after every change.

- Service extraction: the GTFS feed (about 1.4 GB) is filtered to one Thursday peak, with link traversal times derived from the shapes.
- Scenario construction: two Flatland levels and two timetables, the corridor (87 x 8, 52 movements) and the junction (20 x 20, 126 movements).
- MDP construction: level, timetable and disruption regime, with a tree observation, a per-train discrete action and the metric potential as reward.
- Vectorisation and training: two copies of the fleet for PPO, one for A2C and DQN, one sub-environment per train, shared policy parameters.
- Checkpoint selection: PPO is scored every 100k steps on unseen seeds 9001 to 9004 (thirteen evaluations); A2C and DQN every 400k steps on two seeds, completion first.

- Paired evaluation and reporting: the corridor arm and its baseline on the same 30 held-out seeds (3000 to 3029), with an exact sign test.
- Decision audit: every visit to a commit crossover is recorded with whether a blocked cell lay ahead, so the crossings can be shown to be conditional.

2.2 System Architecture Design

One shared multi-layer perceptron controls every train: 1032 inputs, two hidden layers (256 units for PPO, 128 for A2C and DQN) and a five-way action head. The observation is a flattened, normalised tree of depth 3 within a radius of 50 cells, plus twelve conflict and geometry features: slack and its rank, distance to the first occupied cell, distance to oncoming traffic, malfunctioning trains on my route, my own malfunction countdown, current speed, whether I am on the map, remaining route length, queue depth, distance to the nearest blocked cell ahead, and distance to my next commit crossover. The last two exist because a zeroed rail cell is invisible to occupancy features, so without them the crossing decision is not expressible.

The reward is the assessment metric paid densely as a potential, r(t) = R(t) - R(t-1), where R(t) is what the metric would charge if the episode ended at t. The sum telescopes to the metric exactly, verified per agent and per term to machine precision, and training divides the potential by a constant 100 so the value targets stay O(1). All reported arms also use two shaping bonuses on top of the potential: +50 for every train that arrives and -1 for every step a train is stuck, at the same 1/100 scale.

From a random initialisation none of the three algorithms finds the crossing, so the reported corridor arm is PPO fine-tuned from a behaviour-cloned detour baseline (imitation then RL), with the critic warmed up first. The two independent algorithms are the control: they share the observation, reward and regime but not the warm start.

Module	Responsibility
nb_harness.py	Executes the notebook's cells so the CLI tools use exactly the notebook's code
regimes.py	Map geometry (seven commit crossovers) and the regime selector (block by default)
disruptions.py	Both mechanisms: train-borne failures and the exogenous blockage
priority_obs.py	The twelve conflict/geometry observation features (1032-d input)
metric_rewards.py	The metric as a dense potential, plus the shaped variant
train_ppo.py	PPO training, dense reward, checkpoint selection, critic warm-up
train_algo.py	A2C and DQN on the same environment and evaluator
eval_pair.py	Paired evaluation of an arm against its baseline on held-out seeds

Module	Responsibility
bc_warmstart.py	Detour teacher, behaviour-cloning labels, warm-start checkpoint
crossing_audit.py	Score, per-crossover decisions, and regime exposure
render_policy_gif.py	Episode figures, red-X overlay and diagnostic markers
verify_rewards.py	Proves the dense reward reproduces the metric per agent and per term
s2_env.py, s2_assembly.py	Junction environment assembly (rail, fleet, timetable)

Table 2: The modules on a live path. The notebook imports five of them directly; the rest are command-line entry points that execute the notebook's cells through nb_harness.

2.3 GUI/Environment Design

Flatland's own renderer is the interface, which the brief permits in place of a separate GUI: it shows the rail grid with its switches and crossovers, one coloured marker per train, and a red X on every cell that may not be entered - an exogenous blockage on the corridor, a train standing in malfunction on the junction. Using the simulator's renderer means every episode figure comes from the same process the agent is trained in, produced headlessly with the PIL backend. Figure 2 shows the corridor level and Figure 3 the junction.

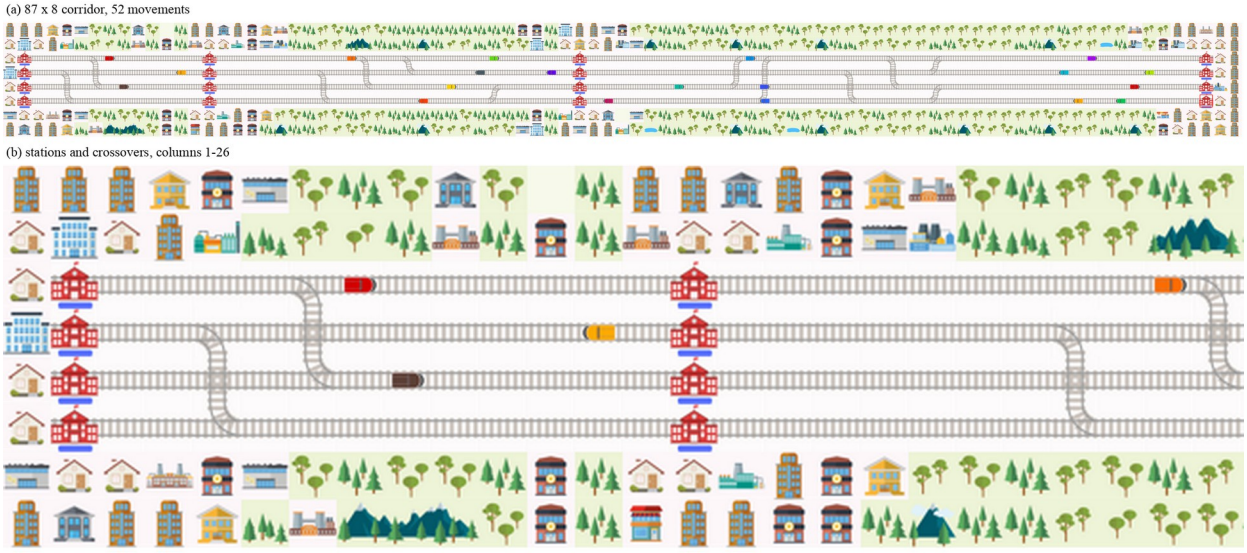


Figure 2: The corridor. (a) The full 87 x 8 level with 52 movements. (b) Columns 1 to 26 at native scale, showing Strathfield, Burwood and the switch ladders.

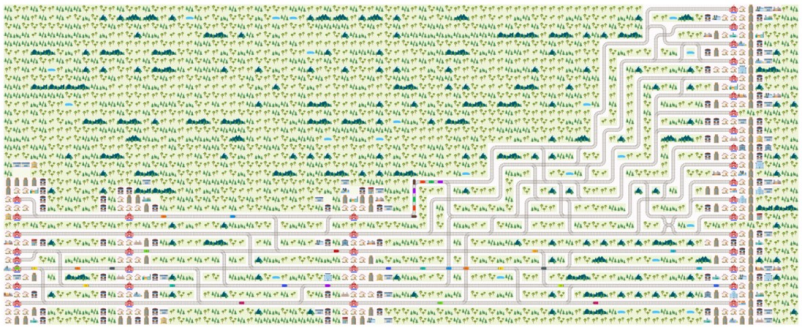


Figure 3: The junction. A 20 x 20 level where the Suburban pair and the Old Main South converge and diverge, carrying 126 scheduled movements.

The disruption regime is the methodological core. A train-borne failure fires when a train reaches a cell, which makes the disruption a function of the policy's own trajectory: a policy that reroutes early can prevent the incident from firing at all. The corridor therefore uses an exogenous blockage: a cell's rail transitions are removed outright at a seed-drawn step and restored after 400 steps, so the failure exists whether or not a train goes near it. Two are drawn per episode, one per direction, from the cells just past each commit crossover; activation is deferred while a cell is occupied, and dropped if it stays occupied for 60 steps. Both arms are given the same cells at the same seed-drawn steps, so the pairing is by seed and blockage plan.

3. Results and Evaluation

3.1 Experimental Settings

Table 1 lists the settings. All arms act deterministically; the corridor arm is evaluated on seeds 3000 to 3029, which training and checkpoint selection never see, and A2C and DQN are reported from their selection seeds only. Because the baseline and the arm run on the same seed, the same environment and the same blockages, a delta is computed per seed and the report gives its mean, median, win count and an exact two-sided sign test. The score is the assessment metric normalised to [-1, 0], with each train's cost capped at one episode, summed over all agents and divided by the number of agents and the episode budget.

The regime and the policy are CPU-bound: Flatland steps the fleet in Python and the 1032-d tree observation is rebuilt for every agent at every step. The shipped corridor run took 1,865 s for 1,224,704 timesteps including thirteen four-episode evaluations (about 660 environment steps per second), and once startup is paid each evaluation interval costs about 148 s, so the evaluator rather than the optimiser sets the wall clock. Training ran on an x86_64 workstation with an RTX 3080 (the system_info recorded in the checkpoint); the notebook and figures also run on an aarch64 relay with the same pinned stack, where the sixteen tests pass. Every table in this report is regenerated from the CSVs in results/ by scripts/build_part_e_report.py; the commands that produced those CSVs are listed in the repository README and in Appendix B of the standalone report.

3.2 Pre-Processing

- Service extraction: one Thursday service period, with rail links rather than straight lines giving traversal times.
- Corridor level: four adjacent through tracks with the ladder rows removed; the compact map leaves a two-cell diversion premium at 7 of the 15 stop types.
- Junction level: a 20 x 20 level from the published track plan, with the turn sequence read out of the timetable as a route dictionary.
- Deadlines and episode budget: calibrated from a reference rollout, with the budget extended past the last arrival because the serialised value cut 15 junction trains off mid-journey.
- Multi-cell targets: the target set is rebuilt from the agent's own last waypoint group after every reset, otherwise a diverted train can never finish.
- Dense reward: the metric as a potential, verified against DefaultRewards per agent and per term.
- Metric hygiene: delay_all replaced a total-delay statistic that went NaN when nothing finished and negative when few did.
- Regime hygiene: the blockage is exogenous, so the disrupted condition cannot collapse into the undisrupted one.

3.3 Experimental Results

The corridor result is the headline. PPO improves the score from -0.0662 to -0.0005, a mean paired gain of +0.0657 with a median of +0.0648 (Table 3). It wins all 30 seeds and every train completes in both arms, so the gain is punctuality rather than cancellation: total lateness falls from 3321 steps to -70. The on-time share moves only from 0.625 to 0.635: the gain is in how late the affected trains are, not in which trains complete. Figure 4 shows the per-seed spread.

Arm	Baseline	Arm	Delta	Wins	Sign test p	Completion	delay_all
PPO	-0.0662	-0.0005	+0.0657	30/30	< 1e-4	1.000 → 1.000	3321 → -70

Table 3: Corridor, 30 paired held-out seeds under the block regime. Only PPO has the full paired test; Table 4 reports what the other two algorithms did in training.

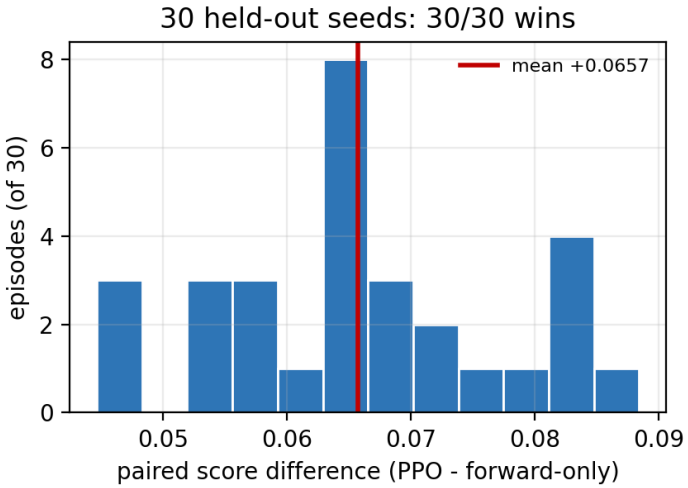


Figure 4: The 30 per-seed differences behind Table 3. All 30 are positive, with a mean of +0.0657 and a median of +0.0648.

The two independent algorithms are the control for the claim that learning the crossing is not automatic, and Figure 6 traces all three arms over the training budget.

Arm	Best checkpoint	Internal score	Completion	Behaviour in Figure 5
PPO (warm start)	745k	-0.0005	1.000	crosses only when blocked
A2C	400k	-0.0563	1.000	no crossing in the panel
DQN (gamma 0.999)	400k	-0.3234	0.529	parks half the fleet

Table 4: Corridor training outcome per arm. Score and completion are the selection-seed numbers, not the 30-seed paired numbers in Table 3; the behaviour column is a reading of the Figure 5 panels, which are rendered with make_env rather than the evaluator.

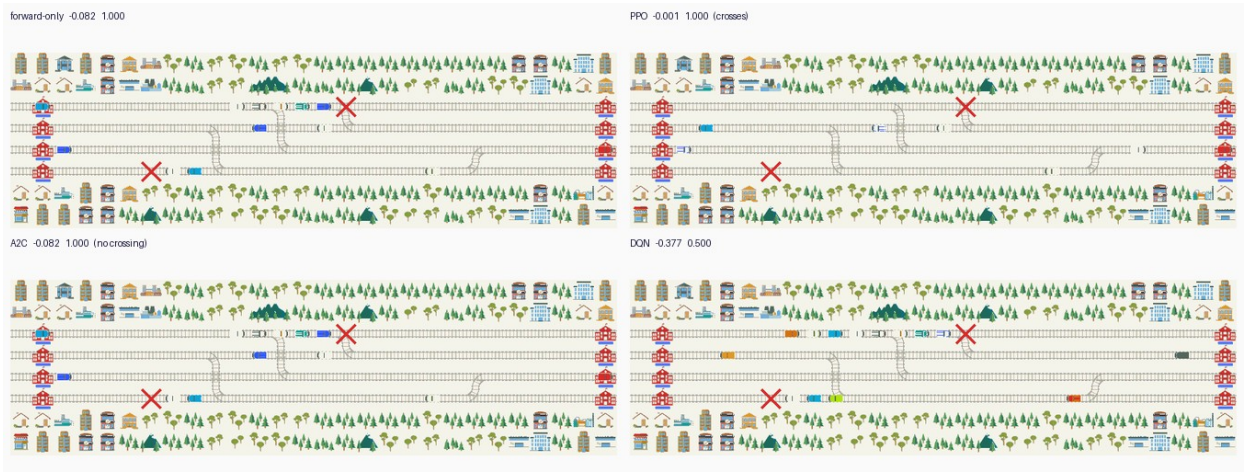


Figure 5: The four corridor arms on the same episode (seed 3001; step 232 for forward-only, PPO and A2C, step 300 for DQN). The red X is a cell whose rail transitions were removed. Forward-only, A2C and DQN leave a queue behind the upper X; PPO's track ahead of it is clear because it changed at the previous crossover.

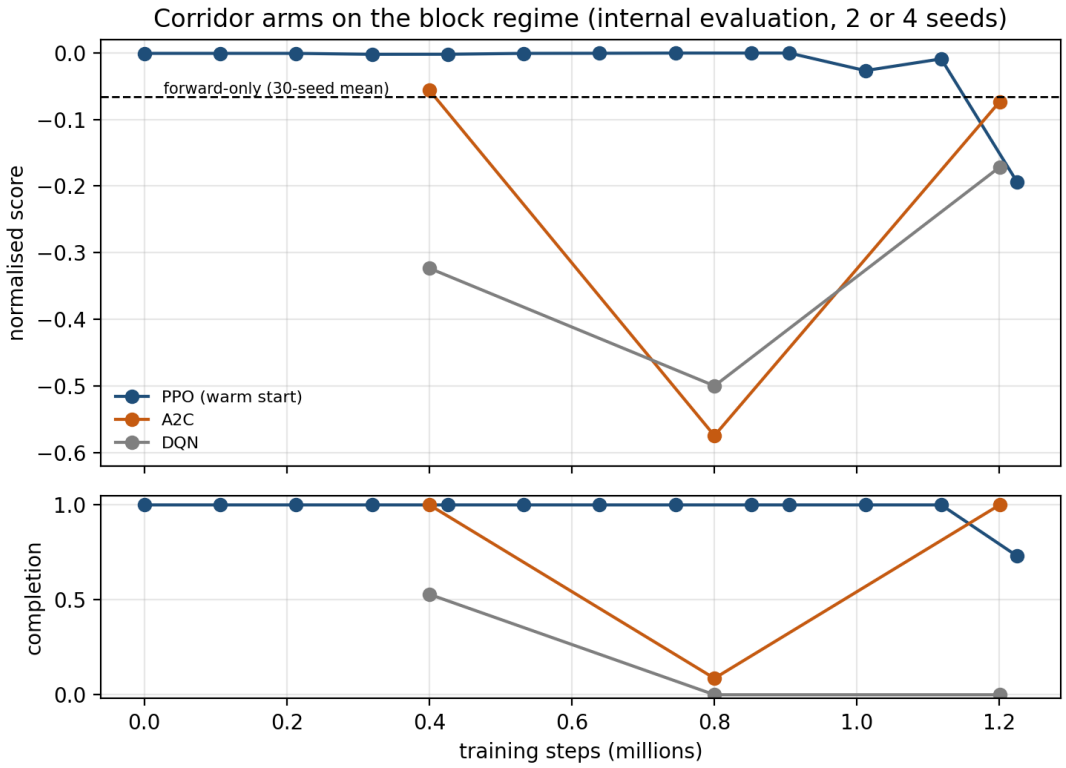


Figure 6: The three corridor arms during training, from the internal evaluation used for checkpoint selection. PPO starts from the behaviour-cloned checkpoint and stays near zero until it drifts at the end of the budget; A2C and DQN fall into the parking basin. The dashed line is the forward-only 30-seed mean.

A score gain alone does not prove the policy changes track for the right reason, since a policy that crossed at every crossover would also gain on this map. The audit counts the decisions themselves: for every agent-step standing on a commit crossover it records whether a blocked cell lay ahead on that train's own row within 60 cells, and whether the agent chose a lateral action. The per-crossover table is shipped as results/crossing_audit_block.csv.

Crossover	Track	Blocked visits	Turns	Open visits	Turns
(2,6)	up_suburban	22	22	30	0
(2,25)	up_suburban	2	2	28	0
(2,62)	up_suburban	0	0	46	0
(4,51)	up_local	18	18	58	0
(5,4)	down_local	0	0	14	0
(5,22)	down_local	19	19	14	0
(5,59)	down_local	40	20	12	0
Total		**101**	**81**	**202**	**0**

Table 5: Per-crossover decision audit over seeds 3000 and 3001. Blocked means an impassable cell ahead on the train's own row within 60 cells. (5,59) is the only crossover with blocked visits that did not always turn (20 of 40); the CSV records counts rather than per-visit causes, so none is claimed.

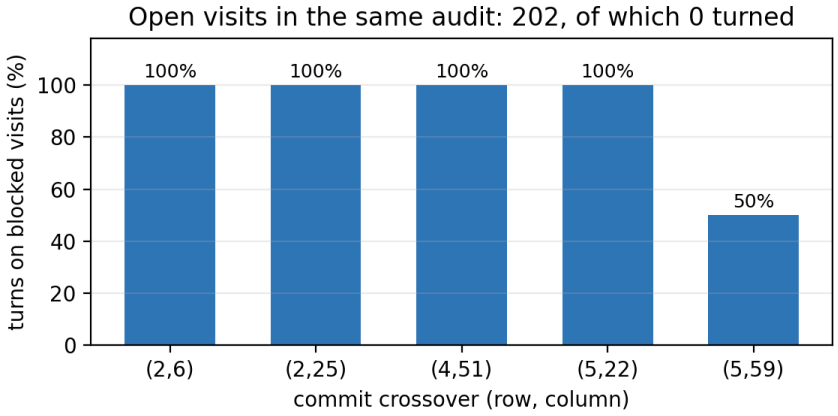


Figure 7: Turn rate on the visits where the track ahead was blocked.

Of 101 agent-steps standing on a crossover with a blocked cell ahead, 81 produced a turn; of 202 with a clear track, 0 did (Figure 7). The audit records one decision per visit to a commit cell, so 81 turns over two seeds is about 40 diverts per episode. The rate is not uniform - four crossovers turn on every blocked visit and (5,59) on 20 of 40 - so the behaviour is state-dependent, not a blanket lane swap, and the A2C and DQN panels of Figure 5 show what the blanket alternative looks like.

Under the clean condition the same checkpoint is inert: on 3 seeds it scores 0.0000, exactly as the forward-only baseline does, and the decision audit records 546 visits to a commit crossover with a clear track and not one turn (results/clean_audit.csv). With the blockage suppressed but the Poisson malfunction layer left on, the arm is indistinguishable from the baseline: -0.0133 against -0.0165, a paired +0.0032 on 3 seeds, 3 of 3 but $p = 0.25$. The crossings in Table 5 therefore belong to the blockage, not to the policy's habit.

Arm	Baseline	Arm	Delta	Wins	Completion (base → arm)
PPO (30 seeds, 1030-d)	-0.1769	-0.0251	+0.1518	30/30	1.000 → 0.000
A2C (30 seeds, 1030-d)	-0.1769	-0.0251	+0.1518	30/30	1.000 → 0.000
DQN (30 seeds, 1030-d)	-0.1769	-0.0208	+0.1561	30/30	1.000 → 0.183
PPO (current, 2 seeds)	-0.1823	-0.2400	-0.0577	0/2	1.000 → 0.468

Table 6: Junction results. The first three rows are the recorded 30-seed runs from before the two observation features were added, kept as measured: that policy parks the fleet. The last row is the current checkpoint selected by completion, which departs and jams; it is a two-seed evaluation shipped as results/junction_current.csv, identically scored on both seeds. The rows are not directly comparable and are labelled accordingly.

The junction is reported as a loss, and its two failure modes bracket the metric's pathology. Parking scores better than attempting: every learned 30-seed arm scores about +0.15 better than the route follower while completing 0 to 18 per cent of the trains, because a cancellation costs about 19 cost units while a dispatch into the jam is charged its full delay. The current checkpoint shows the other side: selected by completion rather than score, it sends every dispatched train into the throat, completing 59 of 126 and scoring -0.2400, worse than the scripted baseline at -0.1823 with 126 of 126 home. Figure 8 sets the two behaviours side by side. Neither is a working junction policy; a global controller, or imitation of the route follower, is what that map needs.

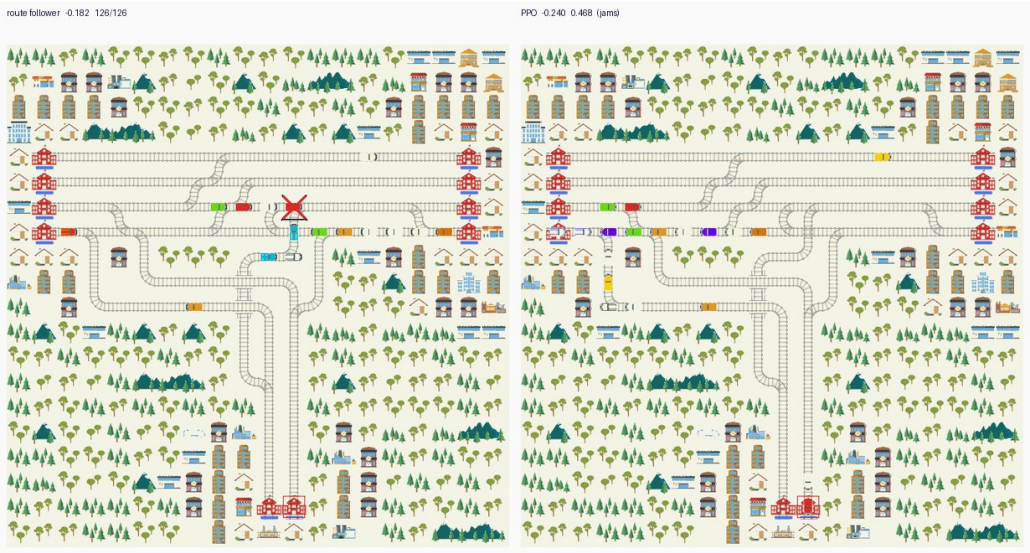


Figure 8: The junction at step 156 under the train-borne regime, rendered with make_env. Left: the scripted route follower, with the held train marked. Right: the selected PPO checkpoint, which dispatches into the throat and jams.

3.4 Limitations

- Only PPO has the full 30-seed paired test on the corridor. A2C and DQN are reported from their selection seeds and the rendered episode; a paired run for them would strengthen the three-algorithm claim and is the obvious next experiment.

- The corridor decision is local: one crossover per blocked stretch and a two-cell detour. The result is not evidence that per-agent RL can arbitrate a junction.
- The disruption volume is a modelling choice, not data: 2 x 400 steps keeps completion at 1.000 while making every affected trip late.
- From-scratch learning does not find the crossing, and the A2C and DQN arms demonstrate it independently. The corridor arm is imitation-then-RL, not pure exploration, and the warm start inherits the teacher's osn score of -0.0005.
- The 30 held-out seeds vary the blockage placement only: fleet, timetable and deterministic actions are fixed, a single training seed per arm was run, and the mean delta has a sign test but no confidence interval.
- The junction table mixes two observation versions; the 30-seed checkpoints predate the two feature additions and no longer load. They are reported as measured and labelled.
- Evaluation is entirely simulated. The timetable and the track layout are real; there is no operational validation and no human factors layer.

4. Discussion and Conclusions

A local policy can learn to reroute, but only if the evaluation makes the decision real, and only with a recipe that can find a sparse action. On a four-track corridor cut by 400-step exogenous blockages, PPO changes track at the commit crossover just before the blockage and turns a paired deficit of -0.0662 into -0.0005 over 30 held-out seeds, winning every seed without losing a train. The same checkpoint does nothing when nothing is blocked, and an audit of 303 crossover visits shows that its crossings follow the blockage rather than the map. The two independent algorithms do not get there, which makes the corridor result a statement about the recipe as much as about the algorithm.

The result depended on fixing the measurement first. A train-borne failure fires when a train arrives, so a policy that reroutes suppresses the incident, and a failure placed deep inside a station's escape band leaves every follower already past the crossover. Removing the cell's transitions instead makes the blockage exogenous, keeps both arms on the same incidents, and turns the delayed comparison into a genuine decision.

Three extensions follow. First, the A2C and DQN arms deserve the same 30-seed paired run; the current evidence is their selection seeds and the rendered episode. Second, the junction needs a global view - a graph or attention observation, or imitation of the route follower - because a local tree observation cannot arbitrate 126 trains through one throat. Third, the corridor result should be checked against the stronger scripted distance-map baseline, not only the forward-only reflex, to see how much of the gain survives when the baseline is allowed to divert too.

5. References

Gorrey, M., Segaert, A., Ganko, J., & Kaine, E. (2025, May 21). 'Delay all non-essential travel': Commuters warned to expect major delays. The Sydney Morning Herald.

Li, B., Guo, T., Li, R., Wang, Y., Ou, Y., & Chen, F. (2021). Delay propagation in large railway networks with data-driven methods. Physica A: Statistical Mechanics and its Applications, 583, 126305.

Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., & Thadden, T. (2020). Flatland: a framework for reinforcement learning on train scheduling. Zenodo / AIcrowd.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: reliable reinforcement learning implementations. Journal of Machine Learning Research, 22(268), 1-8.

Roberts, M. (2024, June 17). Signal repairs cause train disruption in Sydney during Monday morning commute. ABC News.

Semrov, D., Marsetic, R., Zura, M., Todorovski, L., & Srdic, A. (2016). Reinforcement learning approach for train rescheduling on a single-track railway. Transportation Research Part B, 86, 250-267.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

Sydney Trains. (2025). Sydney Trains Annual Report 2025. Transport for NSW.

Transport for NSW. (2026). Public transport timetables and GTFS feeds (Open Data). https://opendata.transport.nsw.gov.au

Zhu, Y., & Goverde, R. M. (2019). Railway timetable rescheduling with flexible stopping and flexible short-turning during disruptions. Transportation Research Part B, 123, 149-181.

Appendices

A. Individual Contribution

Each group member writes up to half a page on their contribution to the project development activities.

Daniel James Martirosov (24948933)

GTFS extraction and both scenarios; the disruption regimes (band, crossing and the exogenous blockage) and the decision they forced; PPO and A2C/DQN training, the behaviour-cloned warm start, the paired evaluator and the crossing audit; the report and template build scripts.

Ziheng Deng (25595315)

[To be completed by the member: 100-150 words on your contribution.]

Laurean Lim (25049863)

[To be completed by the member: 100-150 words on your contribution.]

B. Individual Contribution Split

Team Member Name	Percentage
Daniel James Martirosov	33.33%
Ziheng Deng	33.33%
Laurean Lim	33.33%

Table 7: Agreed contribution split.

All the team members have discussed and agreed on the above individual contribution to the project.

Daniel James Martirosov : _____

Ziheng Deng : _____

Laurean Lim : _____